

Plataforma de ticketing de alta demanda

Documento tecnico de arquitectura

Version	1.0
Fecha	Julio de 2026
Estado	Propuesta para revision de ingenieria
Mercado objetivo	Estados Unidos
Audiencia	Equipo de ingenieria y arquitectura

Contenido

Contenido	2
1. Resumen ejecutivo	3
2. Principios de diseno	3
3. Vista general de la arquitectura	3
4. Tecnologias seleccionadas	4
4.1 TanStack Start + React 19 - capa de presentacion	4
4.2 Hono - capa de API	4
4.3 Cloudflare Durable Objects - motor de concurrencia	5
4.4 Fila virtual (waiting room) sobre Durable Objects	5
4.5 Cloudflare Queues - procesamiento asincrono	5
4.6 PostgreSQL + Hyperdrive + Drizzle ORM - registro	6
4.7 Workers KV, Cache API y R2 - lectura y estaticos	6
4.8 Turnstile, WAF y Stripe Radar - seguridad y anti-bots	6
4.9 Stripe - procesamiento de pagos	6
4.10 Better Auth - identidad	7
5. Flujo de compra bajo alta demanda	7
6. Ciclo de vida del inventario	8
7. Escalabilidad y limites operativos	8
8. Alternativas consideradas	9
9. Riesgos y mitigaciones	9

1. Resumen ejecutivo

Este documento describe la arquitectura técnica propuesta para una plataforma de venta de boletas para eventos en Estados Unidos, diseñada específicamente para el escenario más exigente del dominio: la salida a la venta (*on-sale*) de un evento de alta demanda, donde decenas de miles de usuarios compiten simultáneamente por un inventario finito de asientos durante los primeros minutos.

La decisión central de diseño es resolver la concurrencia por arquitectura y no por código defensivo: el inventario vive dentro de **Cloudflare Durable Objects**, un primitivo de cómputo con estado cuya ejecución *single-threaded* serializa cada operación de reserva. Con ello, la sobreventa (*oversell*) resulta imposible por construcción, sin locks distribuidos, sin transacciones optimistas y sin una capa de Redis que operar. El resto del sistema se apoya en la plataforma *serverless* de Cloudflare (Workers, Queues, KV, R2), con TypeScript de extremo a extremo, PostgreSQL como registro contable y Stripe como procesador de pagos para el mercado estadounidense.

El documento justifica cada elección tecnológica frente a sus alternativas, describe los flujos de interacción críticos y detalla los límites operativos y las estrategias de escalado.

2. Principios de diseño

Corrección por construcción. Las invariantes críticas del negocio (un asiento se vende exactamente una vez) se garantizan por el modelo de ejecución, no por disciplina del programador. Elegimos primitivas cuya semántica hace imposible el estado inválido, en lugar de detectar y corregir carreras después de que ocurren.

El pico es el caso base. Un *on-sale* multiplica el tráfico normal por varios órdenes de magnitud en segundos. La arquitectura no se dimensiona para el promedio con capacidad de reacción, sino que asume el pico como condición de operación: cómputo que escala a cero y a miles de instancias sin aprovisionamiento, y control de admisión explícito antes del inventario.

TypeScript de extremo a extremo. Un único lenguaje y un único sistema de tipos desde el componente de React hasta la fila de mensajes reduce la fricción de contexto, permite compartir esquemas de validación (Zod) entre cliente y servidor y habilita inferencia de tipos entre capas.

Estado caliente separado del registro. El inventario en disputa (milisegundos, alta contención) y el registro contable (consultas analíticas, reportes, conciliación) tienen requisitos opuestos. Se atienden con almacenes distintos conectados por una fila de eventos, en lugar de forzar una sola base de datos a servir ambos perfiles.

Operación mínima. Todos los componentes son servicios gestionados con escalado automático. No hay clusters que dimensionar, versiones de Redis que parchar ni balanceadores que configurar; el equipo invierte su tiempo en producto.

3. Vista general de la arquitectura

Todo el cómputo de la aplicación se ejecuta en el edge de Cloudflare, sobre isolates V8 distribuidos en más de 300 puntos de presencia. El cliente conversa con el renderizado del lado del servidor de TanStack Start y con la API construida en Hono; la contención de inventario se resuelve en Durable Objects; los efectos secundarios viajan por Cloudflare Queues; y los servicios gestionados externos (Stripe, PostgreSQL, Resend) se integran mediante API y webhooks firmados.

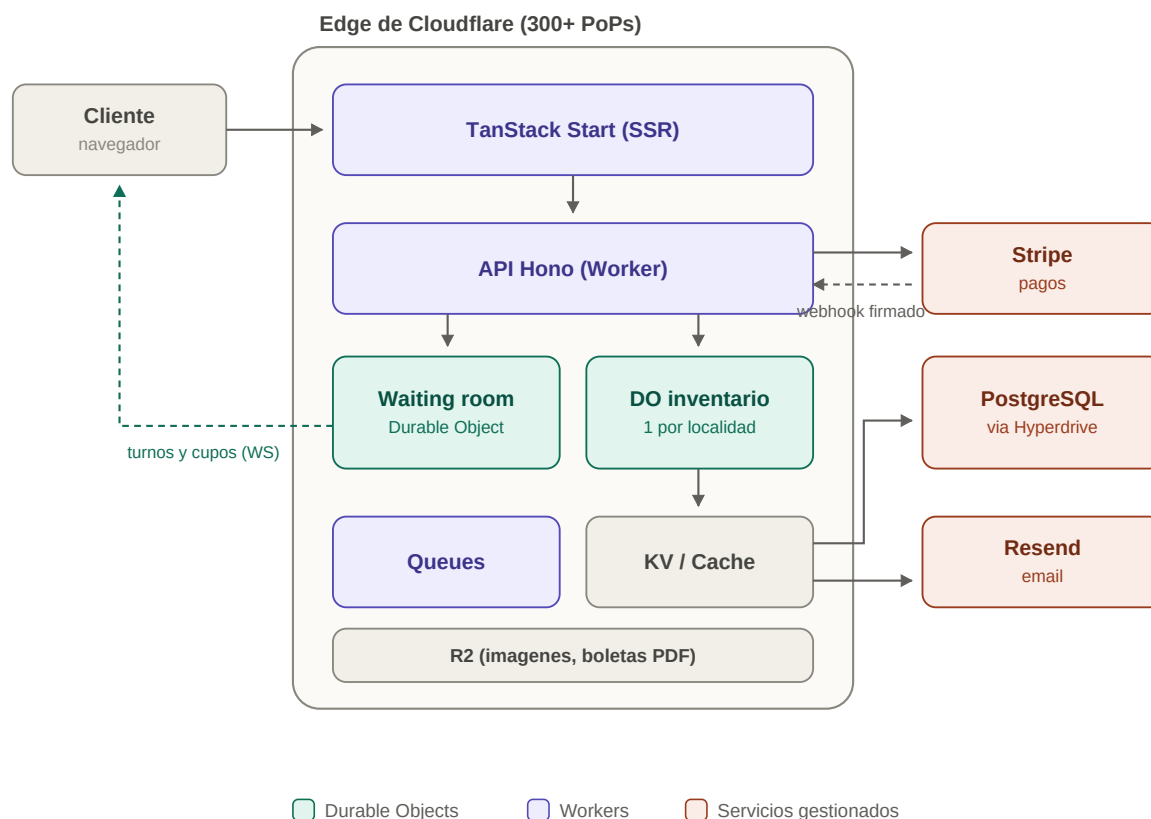


Figura 1. Componentes del sistema y sus relaciones principales.

4. Tecnologías seleccionadas

4.1 TanStack Start + React 19 - capa de presentacion

TanStack Start es un framework full-stack de React construido sobre TanStack Router y Vite. A diferencia de Next.js, no emplea React Server Components: usa SSR de documento completo con hidratación y *server functions* explícitas, lo que elimina la frontera mental entre *use client* y *use server* y produce un modelo de datos mas predecible (loaders por ruta con tipado inferido de extremo a extremo).

Rol en el sistema. Renderiza las paginas de evento en el edge (cacheables globalmente), sirve el mapa de asientos y mantiene la conexión WebSocket que muestra posición en fila y disponibilidad en tiempo real. TanStack Query gestiona el cache de datos del cliente; React Hook Form y Zod cubren formularios y validación compartida con el servidor.

Por que se eligio. Cloudflare es socio oficial del ecosistema TanStack y documenta el despliegue de Start sobre Workers como ruta de primera clase, con scaffolding oficial y soporte del plugin de Vite. Frente a Next.js, evitamos el acoplamiento operativo a Vercel, la complejidad de RSC y los arranques en frio de contenedores: los isolates de Workers inician en milisegundos en el punto de presencia mas cercano al usuario, algo determinante cuando el tráfico llega en avalancha global.

4.2 Hono - capa de API

Hono es un framework HTTP minimalista (menos de 20 KB) diseñado nativamente para runtimes de estándares web como workerd, con enrutamiento por trie de alto rendimiento, middleware componible y un cliente RPC que infiere los tipos de cada endpoint en el frontend sin generación de código.

Rol en el sistema. Expone los endpoints publicos (creacion de holds, consulta de eventos, emision de tokens de turno), recibe y verifica los webhooks de Stripe, y actua como frontera de validacion con esquemas Zod antes de tocar cualquier Durable Object.

Por que se eligio. Express y Fastify presuponen APIs de Node.js y no ejecutan en workerd sin adaptadores; Hono trata el edge como ciudadano de primera clase, comparte el sistema de tipos con el resto del stack y su costo por peticion es despreciable, lo que importa cuando cada milisegundo del hot path se multiplica por miles de peticiones por segundo.

4.3 Cloudflare Durable Objects - motor de concurrencia

Un Durable Object (DO) es la materializacion del modelo de actores en la plataforma de Cloudflare: una instancia unica global, direccionable por nombre, que combina computo JavaScript con una base SQLite embebida y co-localizada (hasta 10 GB por objeto). Su propiedad definitoria es la ejecucion single-threaded con *input gates*: todas las peticiones dirigidas a un mismo objeto se procesan en serie, una a la vez, sin intercalado.

Rol en el sistema. Cada localidad de cada evento es un DO que posee su inventario en SQLite. Reservar asientos es un metodo que lee y escribe de forma sincrona sobre datos locales; mientras se ejecuta, ninguna otra peticion puede observar ni modificar ese inventario. La sobreventa deja de ser un bug posible y pasa a ser un estado inalcanzable. Los *alarms* nativos implementan la expiracion de holds (TTL) sin cron externo, y la API de WebSocket con hibernacion permite difundir disponibilidad a miles de clientes conectados sin pagar computo mientras no hay actividad.

Por que se eligio. Las alternativas trasladan la correccion al codigo de aplicacion. Con Redis, la atomicidad exige scripts Lua, y la expiracion de holds, la fila de espera y la difusion en tiempo real son sistemas adicionales que construir y operar. Con PostgreSQL y bloqueo de filas (*SELECT FOR UPDATE*), miles de transacciones compitiendo por las mismas filas calientes degradan en tormentas de conexiones y esperas de lock precisamente en el peor momento. El DO resuelve las tres necesidades (atomicidad, TTL, tiempo real) con un solo primitivo gestionado, y la co-localizacion de datos y computo elimina la latencia de red del ciclo critico de reserva.

4.4 Fila virtual (waiting room) sobre Durable Objects

El control de admision es la pieza que convierte un pico destructivo en carga ordenada. Un DO por evento actua como sala de espera: encola a los usuarios que superan el desafio de Turnstile, publica su posicion por WebSocket y emite tokens de turno firmados (JWT de vida corta) a un ritmo configurable. Solo las peticiones con token valido alcanzan el inventario.

Por que se eligio. El producto Waiting Room de Cloudflare es exclusivo del plan Enterprise; construirlo sobre DOs replica el mismo patron con control total sobre la logica de admision (prioridades, limites por usuario, ritmo por localidad) y a costo marginal. El beneficio arquitectonico es doble: la experiencia del usuario es una fila transparente en vez de errores 500, y el DO de inventario recibe un caudal acotado y predecible en lugar del pico crudo.

4.5 Cloudflare Queues - procesamiento asincrono

Queues es la fila de mensajes gestionada de la plataforma, con entrega *at-least-once*, procesamiento por lotes, reintentos con backoff y cola de mensajes muertos (DLQ).

Rol en el sistema. Todo lo que no debe bloquear la confirmacion de una venta viaja por Queues: generacion de codigos QR, envio de boletas por email, notificaciones a organizadores, proyeccion de eventos hacia PostgreSQL y metricas. El DO de inventario aplica el patron *outbox*: registra el evento de venta en su propio SQLite dentro de la misma operacion que confirma el hold, y lo publica a la fila inmediatamente despues, garantizando que ninguna venta confirmada pierda sus efectos secundarios. Los consumidores son idempotentes por clave de evento, como corresponde a una entrega *at-least-once*.

Por que se eligio. Frente a SQS o Kafka, Queues vive en la misma plataforma y modelo de facturación, se consume con Workers sin infraestructura adicional y su semántica es suficiente para el volumen del dominio. Kafka en particular introduciría un costo operativo desproporcionado para un patrón que aquí es simple distribución de trabajo.

4.6 PostgreSQL + Hyperdrive + Drizzle ORM - registro

PostgreSQL (gestionado, por ejemplo Neon o Supabase) es la fuente de verdad relacional del negocio: catálogo de eventos, ordenes, clientes, liquidaciones a organizadores, contabilidad y reporteria. Hyperdrive es el servicio de Cloudflare que hace viable hablar con Postgres desde el edge: mantiene un pool de conexiones cercano a la base y cachea el handshake TCP/TLS, eliminando el costo de abrir una conexión por petición desde cientos de puntos de presencia. Drizzle ORM aporta SQL tipado sin capa de abstracción pesada y un flujo de migraciones versionadas.

Por que se eligio. Los DOs son excelentes para estado caliente particionado, pero las consultas transversales (ventas por organizador, conciliación con Stripe, business intelligence) requieren un motor relacional con el ecosistema de Postgres. La escritura llega exclusivamente por los consumidores de Queues en lotes, de modo que la base nunca está en el hot path del on-sale y su dimensionamiento es independiente del pico.

4.7 Workers KV, Cache API y R2 - lectura y estáticos

KV es un almacén clave-valor replicado globalmente con consistencia eventual, ideal para datos que se leen millones de veces y cambian poco: metadatos de eventos, configuración de venta, feature flags. La Cache API de Workers permite cachear respuestas HTML completas de las páginas de evento en cada punto de presencia. R2 es el object storage compatible con S3 y sin costos de egreso, donde viven imágenes de eventos y boletas PDF generadas.

Por que se eligio. Durante un on-sale, la inmensa mayoría de las peticiones son lecturas de la página del evento; servir las desde cache en el edge deja los DOs y la base de datos dedicados exclusivamente al trabajo transaccional. La ausencia de egreso en R2 es relevante en un negocio donde cada boleta vendida implica descargas de PDF e imágenes.

4.8 Turnstile, WAF y Stripe Radar - seguridad y anti-bots

En ticketing el adversario principal no es el atacante clásico sino el bot de reventa, que compete deslealmente con los compradores legítimos. La defensa es en capas: Turnstile (el CAPTCHA invisible de Cloudflare) emite tokens de un solo uso verificados en el servidor antes de admitir a la fila; las reglas de rate limiting del WAF cortan patrones de abuso por IP, ASN y huella; y Stripe Radar evalúa riesgo en el momento del pago, con reglas de velocidad y 3D Secure dinámico para transacciones sospechosas.

Por que se eligio. Turnstile y WAF operan en la misma red que sirve el tráfico, sin latencia adicional ni integración de terceros; Radar viene incluido en Stripe y cierra el último eslabón (fraude de tarjeta), que las capas de red no pueden ver.

4.9 Stripe - procesamiento de pagos

Stripe es el procesador de pagos de referencia para el mercado estadounidense, con SDK oficial en TypeScript que ejecuta en workerd usando su cliente HTTP basado en fetch y verificación de webhooks sobre Web Crypto.

Rol en el sistema. El checkout usa Payment Element embebido sobre PaymentIntents, lo que habilita tarjetas, Apple Pay, Google Pay y Link (pago en un clic) sin salir de la página; en una venta relámpago, cada segundo de checkout se traduce directamente en conversión. La confirmación de negocio ocurre únicamente vía webhook firmado (*payment_intent.succeeded*), nunca por la respuesta del navegador. Si el modelo comercial es de marketplace, Stripe Connect con *destination charges* y *application fee* resuelve el reparto y los payouts a organizadores; Stripe Tax calcula el sales tax por estado, una obligación real del dominio en EE. UU.

Por que se eligio. Ninguna alternativa combina la cobertura de wallets, la calidad del SDK y las capacidades de plataforma (Connect, Tax, Radar) con el mismo nivel de madurez. El hold del inventario es siempre la autoridad: si expira antes del pago, el DO cancela el PaymentIntent, evitando cobros sin boleta.

4.10 Better Auth - identidad

Better Auth es una libreria de autentificacion TypeScript-first que ejecuta en Workers y persiste su esquema en nuestro propio PostgreSQL mediante el adaptador de Drizzle.

Por que se eligio. Soporta de fabrica passkeys y magic links (clave para un checkout sin friccion en dispositivos moviles), y al ser una libreria y no un SaaS, no introduce costo por usuario activo ni dependencia de un proveedor externo para el dato mas sensible del sistema: la identidad de los clientes. Frente a Auth0 o Clerk, el esquema de usuarios vive en nuestra base, versionado con las mismas migraciones que el resto del dominio.

5. Flujo de compra bajo alta demanda

La figura 2 muestra la secuencia completa de una compra durante un on-sale. Tres decisiones estructuran el flujo. Primero, el usuario no toca el inventario hasta portar un token de turno firmado, emitido por la sala de espera a ritmo controlado. Segundo, la reserva y la confirmacion son dos operaciones separadas por el pago: el hold concede un derecho temporal (10 minutos) respaldado por un alarm, y solo el webhook firmado de Stripe lo convierte en venta. Tercero, todos los efectos posteriores a la venta se desacoplan por Queues, de modo que la ruta critica termina en cuanto el DO confirma.

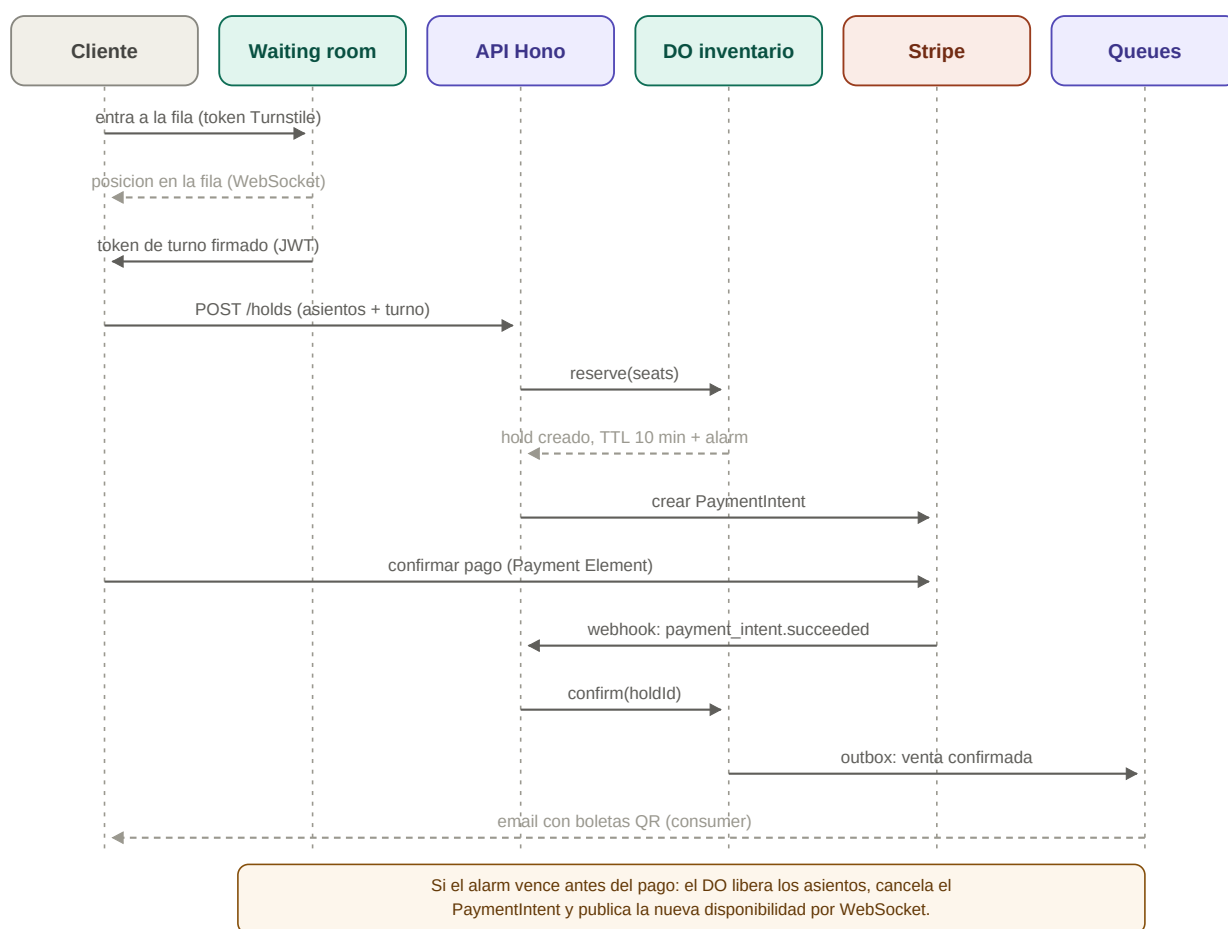


Figura 2. Secuencia de compra durante un on-sale, incluida la expiracion del hold.

La verificación del webhook merece detalle: Stripe firma cada evento con HMAC y el SDK lo valida en workerd con la variante asincrónica basada en Web Crypto. Además, el manejador es idempotente por identificador de evento, de modo que los reintentos de Stripe (o un webhook duplicado) no pueden confirmar dos veces la misma venta ni disparar dos emisiones de boletas.

6. Ciclo de vida del inventario

Cada asiento es una fila en el SQLite de su Durable Object y transita una máquina de estados deliberadamente pequeña. La transición de *libre* a *en hold* ocurre dentro de una única ejecución serializada del DO; la de *en hold* a *vendido* la dispara exclusivamente el webhook de pago; y el regreso a *libre* es responsabilidad del alarm, que además cancela el `PaymentIntent` asociado y difunde la nueva disponibilidad a los clientes conectados. No existe ningún camino que produzca dos ventas del mismo asiento.

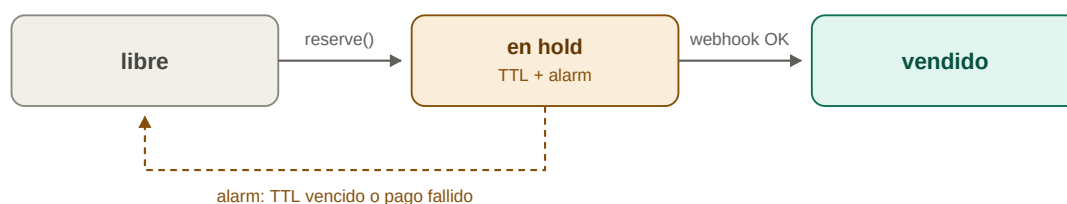


Figura 3. Máquina de estados de un asiento dentro del Durable Object de su localidad.

El TTL de 10 minutos equilibra dos fuerzas: suficiente para completar un pago con wallet o tarjeta (incluido un desafío 3D Secure), y suficientemente corto para que el inventario retenido por carritos abandonados regrese rápido a la venta durante el pico.

7. Escalabilidad y límites operativos

El límite honesto del diseño es el propio Durable Object: al ser single-threaded, un objeto procesa del orden de cientos de operaciones de escritura por segundo. Dos mecanismos convierten ese límite en un no-problema. El primero es el particionamiento: la unidad de inventario es la localidad, no el evento, de modo que un estadio se reparte entre tantos objetos como localidades tenga y la contención se divide con el (figura 4). El segundo es el control de admisión: la sala de espera garantiza que el caudal hacia cada objeto nunca exceda su capacidad, transformando el exceso en fila ordenada en lugar de saturación.

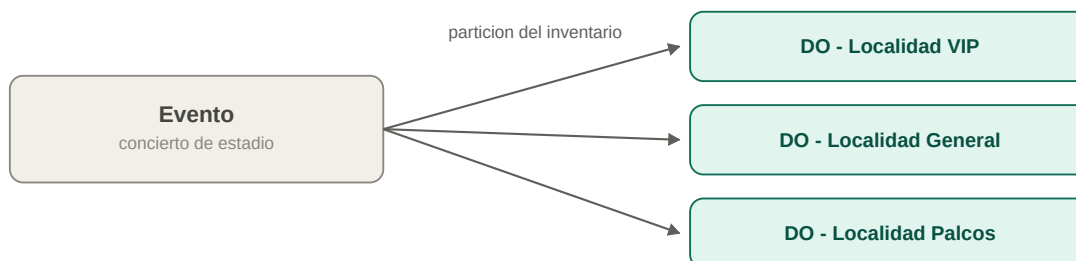


Figura 4. Particionamiento del inventario por localidad: la contención escala con el número de objetos.

Las lecturas siguen el camino inverso: la disponibilidad se difunde por WebSocket desde el propio DO (push, no polling), las páginas de evento se sirven desde cache en el edge y el catálogo desde KV. El resultado es que el tráfico masivo de lectura jamás compite con las escrituras de reserva. PostgreSQL, por su parte, recibe escrituras en lotes desde los consumidores de Queues y puede dimensionarse para el promedio, no para el pico.

Los 10 GB de SQLite por objeto superan en varios ordenes de magnitud lo que un mapa de asientos requiere.

8. Alternativas consideradas

Las siguientes opciones se evaluaron y se descartaron para este sistema. Varias son tecnologías excelentes en otros contextos; el criterio fue exclusivamente su ajuste al problema de inventario contenido bajo picos extremos con un equipo pequeño.

Alternativa	Motivo del descarte
Redis / Upstash como inventario	La atomicidad exige scripts Lua y disciplina; TTLs, fila de espera y tiempo real son sistemas adicionales que construir. Es el patron clasico, pero con mas piezas moviles y mas superficie de error que un DO, que ofrece las tres propiedades en un solo primitivo gestionado.
PostgreSQL con SELECT FOR UPDATE / SKIP LOCKED	Correcto en teoria; en la practica, miles de transacciones compitiendo por filas calientes generan esperas de lock y tormentas de conexiones justo durante el pico. Ademas exige un pool central que contradice el despliegue en edge.
Next.js sobre Vercel	RSC agrega complejidad sin beneficio para este dominio y el computo con estado (el nucleo del problema) no existe en la plataforma; habria que anadir Redis o similar de todos modos. Costos menos predecibles bajo picos de trafico.
Stack AWS (Lambda + SQS + DynamoDB)	Viable, pero DynamoDB resuelve la atomicidad con escrituras condicionales sin ofrecer serializacion de logica arbitraria ni WebSockets integrados; Lambda anade arranques en frio y la composicion (API Gateway, IAM, VPC) multiplica la complejidad operativa para un equipo pequeno.
Kafka / Redpanda para eventos	Semantica y throughput muy por encima de lo que el dominio necesita, a cambio de un costo operativo permanente. Queues cubre distribucion de trabajo con reintentos y DLQ sin infraestructura propia.

9. Riesgos y mitigaciones

Riesgo	Mitigacion
Saturacion de un DO en un on-sale extremo	Particionamiento por localidad (y sub-localidad si hiciera falta) mas control de admision en la sala de espera: el caudal hacia cada objeto es un parametro configurable, no una variable del trafico.
Dependencia de la plataforma Cloudflare	Hono, Drizzle, Better Auth, Stripe y PostgreSQL son portables. El acceso a los DOs se encapsula tras una interfaz de dominio (reservar, confirmar, liberar), acotando a un modulo el costo de una eventual migracion del motor de inventario.
Webhooks duplicados o fuera de orden	Verificacion de firma, idempotencia por identificador de evento persistida en el DO y transiciones de estado que rechazan confirmaciones sobre holds inexistentes o vencidos.
Bots de reventa sofisticados	Defensa en capas (Turnstile, WAF, huella, Radar) mas limites por usuario autenticado y por medio de pago. Se asume mejora continua: es una carrera armamentista, no un problema que se cierra.
Perdida de efectos secundarios de una venta	Patron outbox en el DO (el evento se persiste junto con la venta) y entrega at-least-once en Queues con DLQ y alertas sobre mensajes muertos.

Conclusion. La arquitectura propuesta concentra la dificultad real del dominio (conurrencia sobre inventario finito) en un primitivo disenado exactamente para ese problema, y delega todo lo demas en servicios gestionados de un mismo ecosistema. El resultado es un sistema donde la correccion no depende de la carga, el

costo sigue al uso y el equipo de ingeniería opera producto, no infraestructura.